SYSTEM DOCUMENTATION

Event Registration System

A web-based system for creating events and allowing participants to register online, built on the MVC pattern with Node.js, Express.js, Sequelize and EJS over MySQL or PostgreSQL.

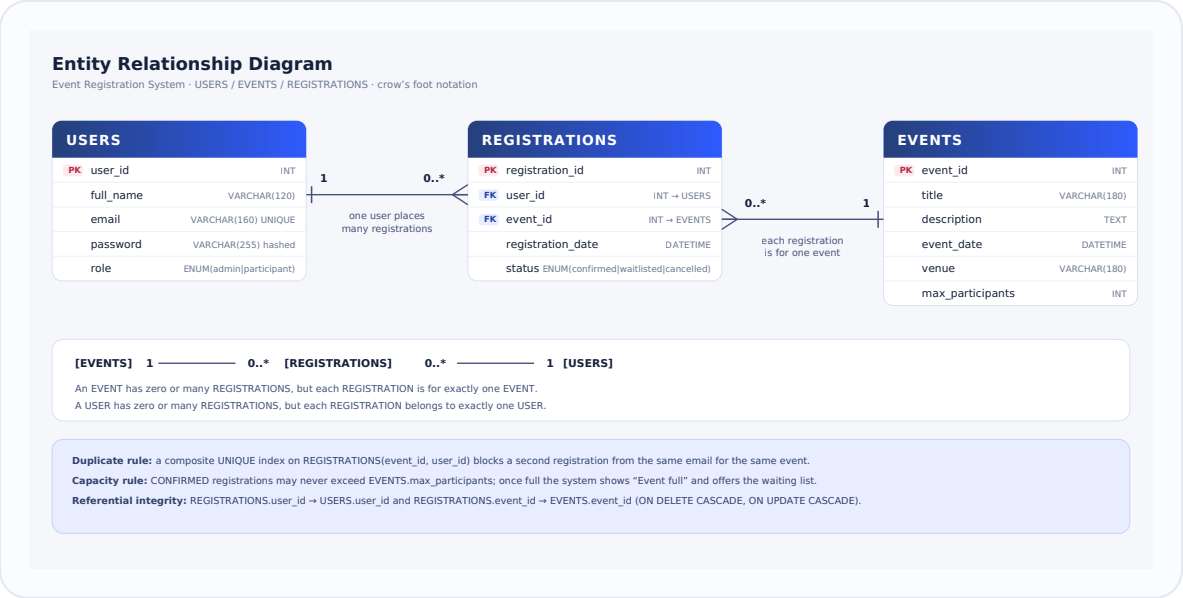
- Entity Relationship Diagram
- System Flowchart
- Feature Specification
- System Requirements
- MVC Architecture

1. System Requirements

REQUIREMENT	VERSION	PURPOSE
Node.js	`18 or later (tested on 20 LTS)`	JavaScript runtime for the server
Express.js	`^4.19`	HTTP server, routing and middleware
Sequelize	`^6.37`	ORM: models, associations, validations, transactions
MySQL or PostgreSQL	`MySQL 8 / PostgreSQL 14+`	Relational database management system
Database driver	`mysql2 or pg + pg-hstore`	Sequelize dialect driver
EJS / HTML / CSS / JS	`EJS ^3.1`	Server-side templating and the browser UI
Modern web browser	`Chrome, Edge, Firefox, Safari`	Client access
Supporting packages	`express-session, method-override, bcryptjs, dotenv`	Sessions, PUT/DELETE from forms, hashing, config

Install & run: `npm install` → `cp .env.example .env` → `npm run seed` → `npm start`, then open `http://localhost:3000`. Default admin: `admin@eventreg.test / admin123`. SQLite is the default dialect so the app runs with no database server; switch to MySQL or PostgreSQL with two lines in `.env`.

2. Entity Relationship Diagram (ERD)



Source: `docs/ERD.svg` · raster copy `docs/ERD.png` · crow's foot notation

2.1 Relationship notation

```
[EVENTS] 1 ——— 0..* [REGISTRATIONS] 0..* ——— 1 [USERS]

• One EVENT has zero or many REGISTRATIONS; each REGISTRATION is for exa
• One USER has zero or many REGISTRATIONS; each REGISTRATION belongs to
• REGISTRATIONS is the associative entity that resolves the many-to-many
  relationship between participants and events.
```

2.2 Data dictionary

USERS

FIELD	TYPE	KEY / CONSTRAINT	DESCRIPTION
user_id	INT, auto increment	**PK**	Surrogate primary key
full_name	VARCHAR(120)	NOT NULL	Complete name of the user
email	VARCHAR(160)	UNIQUE, NOT NULL	Login name; also the duplicate-registration key
password	VARCHAR(255)	NOT NULL	Bcrypt hash - never stored in plain text
role	ENUM(admin, participant)	default participant	Access level of the account

EVENTS

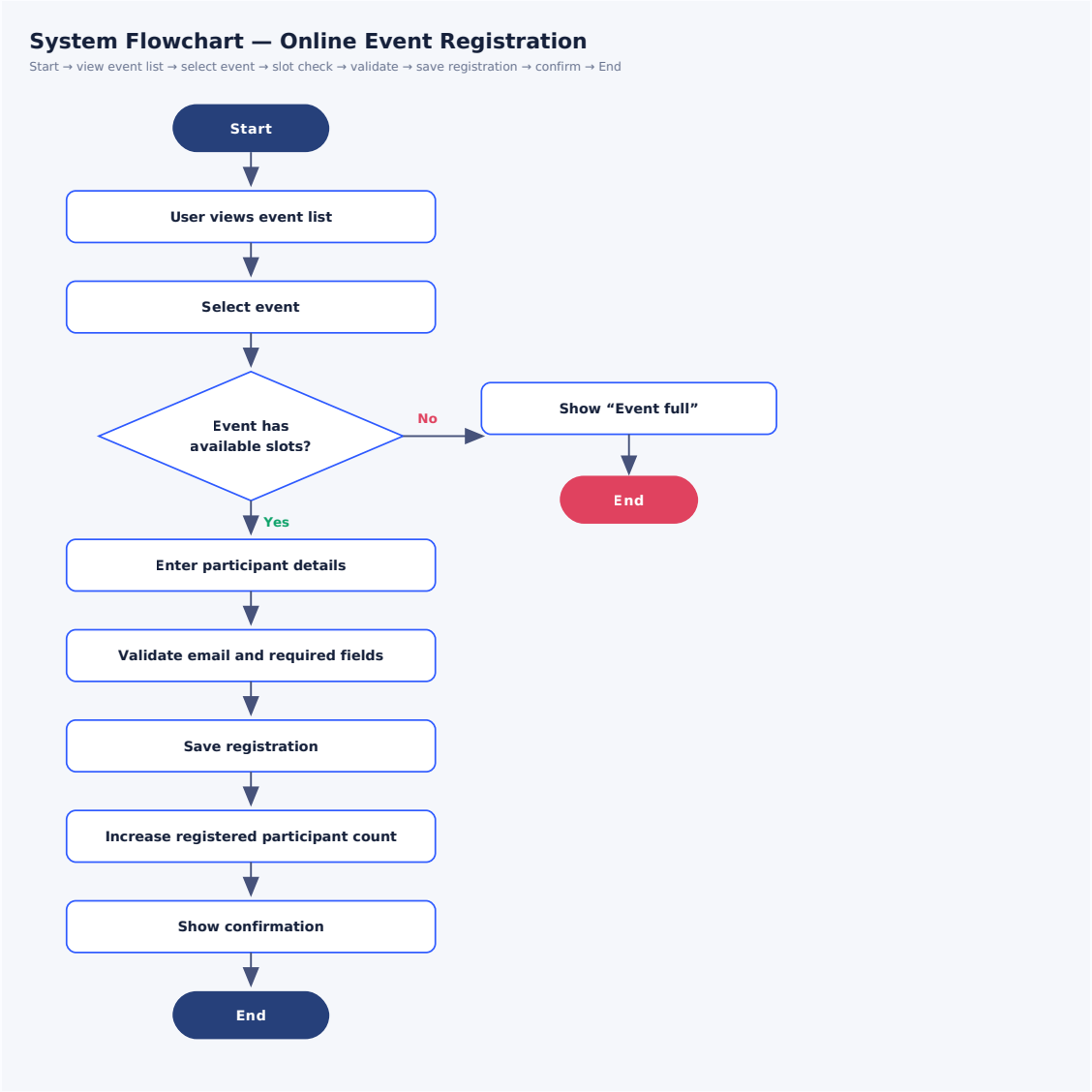
FIELD	TYPE	KEY / CONSTRAINT	DESCRIPTION
event_id	INT, auto increment	**PK**	Surrogate primary key
title	VARCHAR(180)	NOT NULL	Event name on the public listing
description	TEXT	NULL	Full details of the event
event_date	DATETIME	NOT NULL	Date and time of the event
venue	VARCHAR(180)	NOT NULL	Where the event is held
max_participants	INT	NOT NULL, >= 1	Slot limit; registration is blocked once reached

REGISTRATIONS

FIELD	TYPE	KEY / CONSTRAINT	DESCRIPTION
registration_id	INT, auto increment	**PK**	Surrogate primary key
user_id	INT	**FK** -> USERS	Participant who registered
event_id	INT	**FK** -> EVENTS	Event being registered for
registration_date	DATETIME	default now	Timestamp when the registration was saved
status	ENUM(confirmed, waitlisted, cancelled)	default confirmed	Registration state
(composite)	UNIQUE(event_id, user_id)	index	Database-level duplicate prevention

Duplicate rule: a composite UNIQUE index on `REGISTRATIONS(event_id, user_id)` blocks a second registration from the same email for the same event.
Capacity rule: confirmed registrations may never exceed `EVENTS.max_participants`.

3. System Flowchart



Source: `docs/FLOWCHART.svg` · raster copy `docs/FLOWCHART.png`

3.1 Text form and code mapping

```
Start
  |
User views event list                  ->  GET /
  |
Select event                           ->  GET /events/:id
  |
Event has available slots? -- No --> Show "Event full" --> End
  |                                 (HTTP 409, views/events/full.ejs)
 Yes
  |
Enter participant details              ->  POST /events/:id/register
  |
Validate email and required fields     ->  middlewares/validate.js
  |
Save registration                      ->  Registration.create() in a tr
  |
Increase registered participant count  ->  COUNT(status = 'confirmed') r
  |
Show confirmation                      ->  views/events/confirmation.ejs
  |
 End
```

3.2 Exception paths

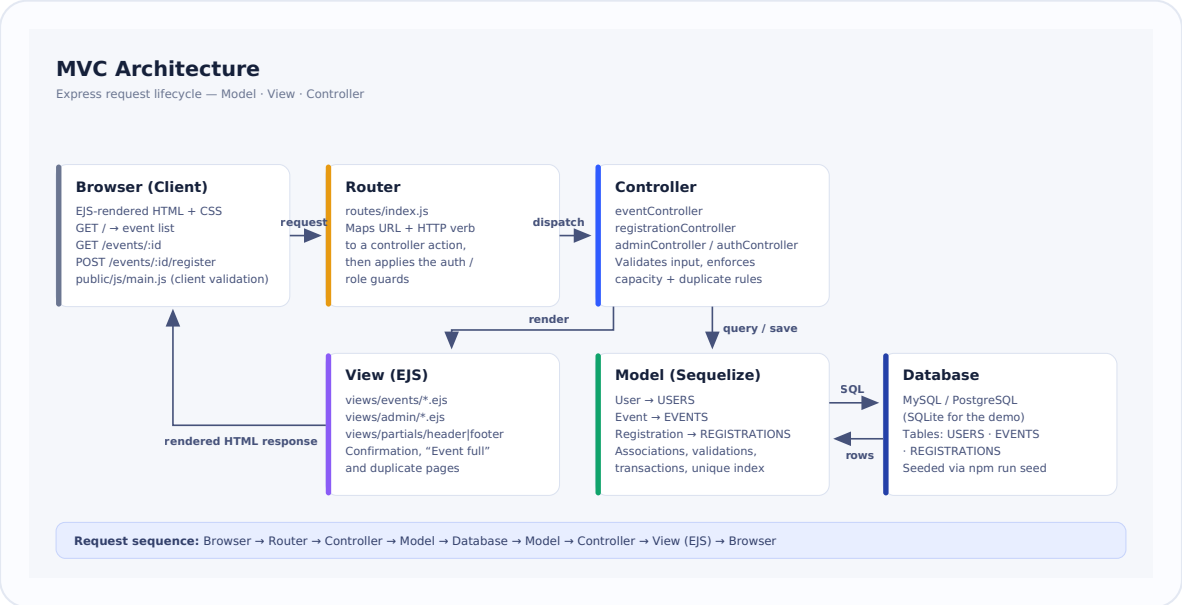
CONDITION	SYSTEM RESPONSE
Email already registered for the event	HTTP 409 + `views/events/duplicate.ejs` — “You are already registered”
Missing field or invalid email	HTTP 422 + form redisplayed with the error list
Two participants submit the last slot at the same time	The second transaction re-counts inside a row lock and returns the “Event full” page — no overbooking
A confirmed participant cancels	Slot is freed and the earliest `waitlisted` registration is promoted to `confirmed`

4. Feature Specification

FEATURE	ROUTE	CONTROLLER ACTION	VIEW / OUTPUT
Create, edit and delete events	`GET/POST /admin/events`, `PUT/DELETE /admin/events/:id`	eventController.create / update / destroy	admin/event-form.ejs, admin/events.ejs
Public event listing page	`GET /`	eventController.publicList	events/index.ejs with search and capacity bars
Online participant registration	`POST /events/:id/register`	registrationController.store	events/confirmation.ejs with reference no.
Limit registrations (max participants)	checked on submit	capacity check inside a transaction	events/full.ejs — “Event full”
Prevent duplicate registration (same email)	checked on submit	duplicate check + UNIQUE index	events/duplicate.ejs
View registered participants per event	`GET /admin/events/:id/participants`	adminController.participants	admin/participants.ejs with status filters
Generate attendance list	`GET /admin/events/:id/attendance` and `.../attendance.csv`	adminController.attendance / attendanceCsv	admin/attendance.ejs printable sheet + CSV
Waiting list (extension		registrationController.waitlist	status `waitlisted`,

FEATURE	ROUTE	CONTROLLER ACTION	VIEW / OUTPUT
of the “full” branch)	`POST /events/:id/waitlist`		auto-promoted on cancellation
Accounts and sessions	`/register`, `/login`, `/logout`, `/my-registrations`	authController	auth/*.ejs, my-registrations.ejs

5. MVC Architecture



Source: `docs/MVC-ARCHITECTURE.svg` · raster copy `docs/MVC-ARCHITECTURE.png`

> **Request lifecycle:** Browser → Router → Controller → Model (Sequelize) → Database → Model → Controller → View (EJS) → Browser

5.1 File-to-layer mapping

LAYER	FILE	RESPONSIBILITY
M	`models/User.js`	USERS model - user_id PK, unique email, bcrypt hook, role
M	`models/Event.js`	EVENTS model - event_id PK, title, description, event_date, venue, max_participants
M	`models/Registration.js`	REGISTRATIONS model - PK, user_id FK, event_id FK, registration_date, status, UNIQUE(event_id, user_id)
M	`models/index.js`	Loads models, defines the 1:0..* associations, capacity helper

LAYER	FILE	RESPONSIBILITY
V	`views/events/index.ejs`	Public event listing page with capacity bars and slot badges
V	`views/events/show.ejs`	Event detail + online registration form
V	`views/events/confirmation.ejs`	Confirmation page with reference number
V	`views/events/full.ejs`	Show "Event full" page (flowchart No branch)
V	`views/events/duplicate.ejs`	Duplicate-registration warning page
V	`views/admin/*.ejs`	Dashboard, manage events, event form, participants, attendance sheet
V	`public/css/style.css`	Responsive styling plus print stylesheet for the attendance list
V	`public/js/main.js`	Client-side required-field and email validation

LAYER	FILE	RESPONSIBILITY
C	`controllers/eventController.js`	Public listing, event detail, create / edit / delete events
C	`controllers/registrationController.js`	Slot check, validation, duplicate check, save in transaction, waitlist, cancel
C	`controllers/adminController.js`	Dashboard, participants per event, attendance list + CSV export
C	`controllers/authController.js`	Sign up, log in, log out, my registrations

LAYER	FILE	RESPONSIBILITY
—	`routes/index.js`	Routing table mapping URLs to controller actions
—	`middlewares/auth.js`	Session auth, admin role guard, flash messages
—	`middlewares/validate.js`	Required-field, email and event validators
—	`server.js`	Express app bootstrap: views, session, routes, error pages, DB sync
—	`seeders/seed.js`	Demo users, 5 events and registrations (npm run seed)
—	`docs/schema.sql`	MySQL DDL with the constraints and reporting queries

6. Route / Endpoint Reference

METHOD	PATH	ACCESS	CONTROLLER ACTION	PURPOSE
`GET`	`/`	public	`eventController.publicList`	Public even listing page (search ?q=
`GET`	`/events/:id`	public	`eventController.show`	Event detai + registration form
`POST`	`/events/:id/register`	public	`registrationController.store`	Save an online registration
`POST`	`/events/:id/waitlist`	public	`registrationController.waitlist`	Join the waiting list when full
`POST`	`/registrations/:id/cancel`	user	`registrationController.cancel`	Cancel own registration
`GET/POST`	`/register`	public	`authController`	Create an account
`GET/POST`	`/login`	public	`authController`	Log in
`POST`	`/logout`	user	`authController.logout`	Log out
`GET`	`/my-registrations`	user	`authController.myRegistrations`	Registration of the logged-in user
`GET`	`/admin`	admin	`adminController.dashboard`	Dashboard tiles + late registration
`GET`	`/admin/events`	admin	`eventController.adminIndex`	Manage-events tabl

METHOD	PATH	ACCESS	CONTROLLER ACTION	PURPOSE
`GET`	`/admin/events/new`	admin	`eventController.showCreateForm`	Create-eve form
`POST`	`/admin/events`	admin	`eventController.create`	Save new event
`GET`	`/admin/events/:id/edit`	admin	`eventController.showEditForm`	Edit-event form
`PUT`	`/admin/events/:id`	admin	`eventController.update`	Save event changes
`DELETE`	`/admin/events/:id`	admin	`eventController.destroy`	Delete eve + its registration
`GET`	`/admin/events/:id/participants`	admin	`adminController.participants`	Registered participant per event
`POST`	`/admin/registrations/:id/status`	admin	`adminController.updateStatus`	Confirm / waitlist / cancel a registration
`GET`	`/admin/events/:id/attendance`	admin	`adminController.attendance`	Generate printable attendance list
`GET`	`/admin/events/:id/attendance.csv`	admin	`adminController.attendanceCsv`	Download attendance list (CSV)

7. Business Rules

#	RULE	ENFORCEMENT
BR-1	Registration is only allowed while `confirmed < max_participants`	Application check, re-counted inside a transaction with a row lock
BR-2	One registration per email address per event	`UNIQUE(event_id, user_id)` index + application check (HTTP 409)
BR-3	Full name and a valid email are required	middlewares/validate.js (server) + public/js/main.js (client)
BR-4	Passwords are hashed, never stored in plain text	bcrypt hook in models/User.js
BR-5	Only admins may create, edit or delete events	requireAdmin middleware → HTTP 403
BR-6	Capacity cannot drop below the confirmed head-count	eventController.update validation
BR-7	Cancelling a confirmed slot promotes the earliest waitlisted person	registrationController.cancel and adminController.updateStatus
BR-8	Attendance list contains confirmed registrations only	adminController.attendance (SQL filter on status)

8. Test Cases

TC	SCENARIO	INPUT	EXPECTED RESULT
TC-01	View public listing	GET /	All events with slot badges and capacity bars
TC-02	Open event detail	GET /events/2	Detail page with registration form
TC-03	Valid registration	new name + email	HTTP 201, confirmation page with reference number
TC-04	Duplicate email, same event	same email again	HTTP 409 — “You are already registered”
TC-05	Invalid email	not-an-email	HTTP 422 with the error list
TC-06	Missing required field	blank name	HTTP 422 — Participant name is required.
TC-07	Register when full	event at capacity	HTTP 409 — “Event full”, limit not exceeded
TC-08	Waitlist a full event	full event + new email	HTTP 201, status waitlisted
TC-09	Cancel a confirmed registration	own registration	Slot freed, earliest waitlisted promoted
TC-10	Create event (admin)	valid form	Event appears in the admin list (HTTP 302)
TC-11	Edit event (admin)	`_method=PUT`	Changes saved (HTTP 302)
TC-12	Delete event (admin)	`_method=DELETE`	Event and its registrations removed
TC-13	View participants		

TC	SCENARIO	INPUT	EXPECTED RESULT
		/admin/events/2/participants	Rows with name, email, date, status
TC-14	Attendance list	/admin/events/2/attendance	Printable sheet, confirmed rows only
TC-15	Attendance CSV	/admin/events/2/attendance.csv	CSV download with correct filename
TC-16	Non-admin opens /admin	participant session	HTTP 403 access-denied page
TC-17	Unknown event	GET /events/999	HTTP 404 page
TC-18	Unknown route	GET /nope	HTTP 404 page